

Improving the Build Process of Pallene

Contributor: Ninad Sachania

Mentors: Hugo Gualandi & Luiz Romário Santana Rios

What problem we are solving?

“Pallene is a typed dialect of Lua designed to write code that is fast and interoperable with Lua. Because Pallene modifies the Lua runtime, Pallene can't run on top of a stock Lua installation. It needs a patched version of the Lua interpreter and a handful of runtime libraries written in C. This means that installing Pallene isn't as simple as running "luarocks install", as is the case with many other Lua projects.

The goal of this GSOC project is to design a simpler, more accessible, and more portable process for building and installing Pallene. This may involve vendoring dependencies that are currently spread over multiple Git repositories and developing new installation scripts.”

What did we do?

- Vendored all the dependencies (both Lua and C)
- Simplified the build process
- Removed LuaRocks as a dependency (for building and installing Pallene)
- Rewrote documentation
- Updated the CI

Benefits

- Simplified the build process significantly
- Protection against supply chain attacks
- Fewer dependencies
- Faster CI
- Faster build times
- Improved robustness

Installing Pallene

Pallene requires a special version of Lua, which you will need to build from source. You will also need to install the Luarocks package manager.

Install the special Lua

You must download and compile the Lua from [our other repository](#). This version of Lua is patched to expose some additional C APIs that Pallene needs.

```
git clone https://www.github.com/pallene-lang/lua-internals/  
cd lua-internals  
make guess -j4  
sudo make install  
cd -
```

If you are on Linux and would like the up arrow to work in the Lua REPL, then run `make linux-readline` instead of `make guess`. After Lua is installed, run `lua -v` to check if you have the right version. It needs to say `Lua 5.x.x` with core `API`. If it doesn't have the "with core API", that means you're using the default Lua instead of the special Lua.

Install Pallene Tracer

For debugging purposes, Pallene uses a tool called Pallene Tracer, which is required to trace function calls throughout Pallene and Lua environments. You must clone and compile Pallene Tracer from its repository [pallene-tracer](#). You need to use a specific version, `0.5.0a`.

```
git clone https://www.github.com/pallene-lang/pallene-tracer --depth 1 --branch 0.5.0a  
cd pallene-tracer  
make LUA_PREFIX=/usr/local  
sudo make install
```

Here, `LUA_PREFIX=/usr/local` is used to point to the patched Lua for Pallene.

Install Luarocks

The next step is to get the Luarocks package manager. Because we built our Lua from source, we must also [build Luarocks from source](#). You can't download Luarocks from your Linux distro, because that would use the wrong version of Lua. To build Luarocks, unpack the sources and run `configure`, `make`, and `make install`. In the configure step, use `--with-lua` to point to our special Lua.

Building and Installing Pallene

Before you are able to build and install Pallene, you'll need to run the configure script.

You can also, optionally, modify the installation location via the configure script. See `./configure --help` for details. The default location is `/usr/local`.

```
./configure  
make
```

To install Pallene and all of its dependencies:

```
sudo make install
```

Using Pallene

Installing Pallene will put two binaries—`pallenecc` and `pallene-lua`—on your `PATH`. `pallenecc` is for compiling Pallene programs and `pallene-lua` is for running Lua programs that load Pallene modules.

To compile a `foo.pln` file to a `foo.so` module:

```
pallenecc foo.pln
```

The resulting `foo.so` can be loaded via the usual `require` mechanism:

```
pallene-lua -l foo
```



njs

H hugomg

I agree it would be nice if the vendoring for argparse worked similarly to lpeg

OK, will update.

BTW, the CI for Pallene Tracer is stuck in some weird loop. Nothing is passing. It's not able to download and install Lua.

Ahh, it looks lua.org itself is down. One more reason for vendoring your dependencies!!! ;-)

Improving other projects

- Fixing a bug in Pallene Tracer

- Improving the configure script for xjump-sdl

Second half

- Integrating Pallene Tracer into our special Lua
- Removing some unnecessary code
- Making Pallene work on macOS, Windows, and WSL2
- Testing portability
- More documentation

Thanks! Any questions?